

Lab 16 - Expected output reference

Pod IPs, ReplicaSet hashes, image digests and ages are runtime values and will differ.

The expectation that matters most

>

On a stock kind cluster, applying default-deny-all **does not block any

traffic**. kindnet, kind's default CNI, does not implement NetworkPolicy

enforcement. Every policy in this lab is accepted and stored by the API server and

then ignored by the data plane.

>

This file contains no transcript of a connection being blocked, because the

training cluster did not block one. The BLOCKED result is shown only as the

expected result on an enforcing CNI, clearly labelled as such.

bash verification/checks.sh - clean run on stock kind (kindnet)

```
== Lab 16 verification: NetworkPolicy and default-deny isolation ==
PASS namespace kcna-lab16 exists
PASS ConfigMap payments-content matches data/payments-index.html byte-for-byte
PASS deployment payments-api is Available and Service payments-api has endpoints
PASS frontend and reporting run the same image and differ by the role label
PASS default-deny-all selects ALL pods ({} ) for both Ingress and Egress
PASS default-deny-all declares no allow rules (isolation by selection)
PASS allow-dns-egress permits UDP/53 and TCP/53 to kube-system
PASS allow-dns-egress ANDs namespaceSelector with podSelector in one peer
PASS allow-frontend-to-payments protects app=payments-api on ingress
PASS allow-frontend-to-payments admits role=frontend only
PASS allow-frontend-to-payments port 8080 equals the payments-api containerPort
PASS allow-frontend-egress-to-payments covers the client side of the same flow
PASS matrix allow: frontend -> payments-api:8080 is expressed by a policy
PASS matrix deny: reporting -> payments-api:8080 is expressed by NO policy
PASS matrix deny: payments-api -> frontend:8080 is expressed by NO policy
NOTE -----
NOTE CNI DaemonSet(s) detected: kindnet kube-proxy
NOTE kindnet does NOT implement NetworkPolicy enforcement.
NOTE Enforcement probe: reporting -> payments-api was REACHABLE.
NOTE That is the EXPECTED result on stock kind and is NOT a policy defect.
NOTE Appendix A shows how to build a kind cluster with Calico for real
NOTE enforcement. The policy OBJECTS above are verified as correct.
NOTE -----
PASS RECORDED: enforcement state of this cluster determined and reported honestly
-- 16 passed, 0 failed --
```

Exit status 0.

Note the design of the last block. The script does not assert BLOCKED. It runs the probe, reports the result, and states plainly why the result is what it is. All fifteen preceding assertions are about the correctness of the policy objects, which is verifiable on any cluster.

bash verification/checks.sh on a cluster with an enforcing CNI

The script detects the CNI and *then* asserts blocking:

```
NOTE -----
NOTE CNI DaemonSet(s) detected: calico-node kube-proxy
NOTE This CNI implements NetworkPolicy, so blocking IS asserted below.
NOTE -----
PASS ENFORCED: frontend -> payments-api is REACHABLE, as the allow rules intend
PASS ENFORCED: reporting -> payments-api is BLOCKED by default-deny-all
PASS RECORDED: enforcement state of this cluster determined and reported honestly
-- 18 passed, 0 failed --
```

A useful real failure on such a cluster, caused by forgetting the client-side egress policy:

```
FAIL ENFORCED: frontend -> payments-api was BLOCKED, expected REACHABLE - check the egress
policy too
```

Key intermediate outputs

The labels that are the whole experiment (Step 1)

NAME	READY	STATUS	RESTARTS	AGE	LABELS
frontend	1/1	Running	0	32s	app=shipper-portal,
role=frontend					
payments-api-5f6d84b7c9-gq2xn	1/1	Running	0	32s	app=payments-api,
tier=backend,...					
reporting	1/1	Running	0	32s	app=finance-reporting,
role=reporting					

The baseline finding (Step 2) - this one *was* observed

```
$ kubectl exec -n kcna-lab16 reporting -- wget -qO- --timeout=5 http://payments-api/ | grep
sensitivity
<p>sensitivity: RESTRICTED - only role=frontend should reach this</p>
```

With no policy anywhere, every Pod can reach every Pod. That is the documented Kubernetes network model, not a misconfiguration.

kubectl describe networkpolicy default-deny-all (Step 3)

```
Spec:
  PodSelector:      <none> (Allowing the specific traffic to all pods in this namespace)
  Allowing ingress traffic:
    <none> (Selected pods are isolated for ingress connectivity)
  Allowing egress traffic:
    <none> (Selected pods are isolated for egress connectivity)
  Policy Types: Ingress, Egress
```

PodSelector: <none> means no constraint - all Pods, not "no Pods". The parenthesised text kubectl prints is the authoritative reading.

kubectl describe networkpolicy allow-frontend-to-payments (Step 4)

```
Spec:
  PodSelector:      app=payments-api
  Allowing ingress traffic:
    To Port: 8080/TCP
    From:
      PodSelector: role=frontend
  Not affecting egress traffic
  Policy Types: Ingress
```

To Port: 8080/TCP is the container port. The Service's port 80 is irrelevant to policy evaluation, because kube-proxy has already rewritten the destination by the time policy is applied.

The AND-versus-OR peer (Step 4)

```
[{"namespaceSelector":{"matchLabels":{"kubernetes.io/metadata.name":"kube-system"}}},
{"podSelector":{"matchLabels":{"k8s-app":"kube-dns"}}}]
```

One list item containing both selectors ☐ AND. Two list items would mean OR and would silently allow egress to *everything* in kube-system.

The enforcement probe (Step 5)

Observed on the stock kind training cluster:

```
--- frontend -> payments-api (policy INTENDS: allow) ---
REACHABLE
--- reporting -> payments-api (policy INTENDS: deny) ---
REACHABLE
```

Expected on an enforcing CNI (Calico/Cilium/Antrea, or a managed cloud CNI) - shown for reference, not observed here:

```
--- frontend -> payments-api (policy INTENDS: allow) ---
REACHABLE
--- reporting -> payments-api (policy INTENDS: deny) ---
BLOCKED
```

Cause, confirmed directly:

```
$ kubectl get daemonsets -n kube-system -o custom-columns='NAME:.metadata.name,IMAGE:.spec.
template.spec.containers[0].image'
NAME          IMAGE
kindnet       docker.io/kindnetd:kindnetd:v20240813-c6f155d6
kube-proxy    registry.k8s.io/kube-proxy:v1.31.0
```

No policy engine is present.

The three-way reasoning check (Step 6)

```
$ kubectl get pods -n kcna-lab16 -l app=payments-api -o name
pod/payments-api-5f6d84b7c9-gq2xn

$ kubectl get pods -n kcna-lab16 -l role=frontend -o name
pod/frontend

$ # policy port vs container port
8080/TCP
8080
```

Destination selector resolves to a real Pod; source selector resolves to exactly the intended client; policy port equals the container port. That review is valid on any cluster and catches most real defects.

Failure-injection reference (Section 6)

```
$ kubectl apply -f manifests/70-networkpolicy-broken-selector.yaml
networkpolicy.networking.k8s.io/allow-frontend-to-payments-broken created
```

Accepted with no warning. describe shows all three defects at once:

```
Spec:
  PodSelector:      app=payments          <- matches ZERO pods (label is app=payments-api)
  Allowing ingress traffic:
    To Port: 80/TCP          <- Service port, not the Pod port 8080
  From:
    PodSelector: role=frontends          <- matches ZERO pods (label is role=frontend)
```

Selectors run as live queries:

```
$ kubectl get pods -n kcna-lab16 -l app=payments
No resources found in kcna-lab16 namespace.

$ kubectl get pods -n kcna-lab16 -l role=frontends
No resources found in kcna-lab16 namespace.
```

And the empty-selector contrast:

```
$ kubectl get networkpolicy default-deny-all -n kcna-lab16 -o jsonpath='{.spec.podSelector}'
{}
$ kubectl get pods -n kcna-lab16 --no-headers | wc -l
3
```

`{}` covers all 3 Pods. A podSelector that matched nothing would produce a "default deny" that denies nothing - the most dangerous silent failure in the whole API, because it fails **open** while looking correct.

Environment caveats recorded for this lab

Caveat	Observed effect	Why
kindnet does not enforce NetworkPolicy	default-deny-all blocks nothing; reporting still reaches payments-api	Enforcement is the CNI's job and kindnet has no policy engine. The API is served regardless, so objects are stored and silently ignored
Nothing reports this	No event, condition, status field or warning; kubectl get netpol is identical on enforcing and non-enforcing clusters	You must inspect the CNI (kubectl get ds -n kube-system) or probe. checks.sh does both
disableDefaultCNI: true is create-time only	Appendix A must build a second cluster to get enforcement	Requires kind create cluster --config, plus internet access to install Calico/Cilium
A CNI-less cluster is NotReady on purpose	Appendix A Step A2 shows NotReady and Pending Pods until a CNI is installed	kubelet reports cni plugin not initialized. This is expected, not a failure
NetworkPolicy ports are Pod ports	The policy says 8080 even though clients dial the Service on 80	Policy is evaluated after kube-proxy's DNAT, below the Service abstraction
Default-deny egress breaks DNS	Every name lookup would fail with bad address on an enforcing CNI	A DNS query is ordinary UDP/TCP 53 traffic to kube-system. allow-dns-egress restores it
namespaceSelector + podSelector placement changes meaning	One list item = AND; two list items = OR	A single misplaced hyphen widens the rule dramatically, with no warning
A default-deny with a non-matching podSelector fails open	Looks applied, protects nothing	Always verify {} or run the selector as a kubectl get pods -l query
Host-network and some external traffic paths	May bypass Pod-level policy depending on CNI and externalTrafficPolicy	Always test from a Pod, not from the node or the host